Email Verification

In the last chapter, we created user signup and login using a unique username and password. In this chapter we'll extend the user authentication by adding email signup to the user model and also add email verification. To do the same, we'll add email field to the model, and once a new user object is created using signup API, we'll create a verification token and send an email to the user with the link to verify the account. We'll also disable user login till the email is verified. First let's add the required fields in the user model.

Browse to users.py in models and add the following lines below password in User class.

```
isVerified = db.Column(db.Boolean,  nullable=False,
default=False)
email = db.Column(db.String(120), unique = True, nullable =
False)
```

And add the following line below username in UserSchema class.

```
email = db.Column(db.String(120), unique = True, nullable =
False)
```

Also since now we have user emails, we'll update find_by_username class method to find by email. So update find_by_username method to the following.

```
@classmethod
def find_by_email(cls, email):
    return cls.query.filter_by(email = email).first()
```

Now your User class should have the following code.

```
class User(db.Model):
    __tablename__ = 'users'
```

```
    id = db.Column(db.Integer, primary_key = True)
    username = db.Column(db.String(120), unique = True,
    nullable = False)
    password = db.Column(db.String(120), nullable = False)
    isVerified = db.Column(db.Boolean,  nullable=False,
    default=False)
    email = db.Column(db.String(120), unique = True, nullable =
    False)
    def create(self):
        db.session.add(self)
        db.session.commit()
        return self
    @classmethod
    def find_by_email(cls, email):
        return cls.query.filter_by(email = email).first()

    @classmethod
    def find_by_username(cls, email):
        return cls.query.filter_by(username = username).first()

    @staticmethod
    def generate_hash(password):
        return sha256.hash(password)

    @staticmethod
    def verify_hash(password, hash):
        return sha256.verify(password, hash)
```

And UserSchema should have the following code.

```
class UserSchema(ModelSchema):
    class Meta(ModelSchema.Meta):
        model = User
        sqla_session = db.session
```

```
    id = fields.Number(dump_only=True)
    username = fields.String(required=True)
    email = fields.String(required=True)
```

Notice here, isVerified field is set to False by default, and once the user verifies the email, we'll set it to True enabling the user to log in.

Next we'll add a util called token.py which will contain methods to generate verification token and confirm the verification token. The verification link in the mail will contain a unique URL with the verification token which should look like `htttp://host/api/users/confirm/<verification_token>` and the token here should always be unique. We'll use itsdangerous package to encode the user email along with a timestamp, so let's go ahead and create token.py in api/utils.

Before we write the code to generate token, we need to add a few more variables to app config since itsdangerous needs a secret key and password salt for it to work which we'll supply from our config.py. Add the following lines in config/config.py under Development, Testing, as well as Production configs ensuring all the keys and salts are different.

```
    SECRET_KEY= 'your_secured_key_here'
    SECURITY_PASSWORD_SALT= 'your_security_password_here'
```

Next, in token.py add the following code to import the requirements.

```
from itsdangerous import URLSafeTimedSerializer
from flask import current_app
```

And then add the following code to generate the token.

```
def generate_verification_token(email):
    serializer = URLSafeTimedSerializer(current_app.
    config['SECRET_KEY'])
    return serializer.dumps(email,salt=current_app.
    config['SECURITY_PASSWORD_SALT'])
```

In the previous method, we use URLSafeTimedSerializer to generate a token using email address, and the email is encoded in the token. Next we'll create another method to validate the token and expiration, and as long as the token is valid and not expired, we'll return the email and verify the user email.

```
def confirm_verification_token(token, expiration=3600):
serializer = URLSafeTimedSerializer(current_app.
config['SECRET_KEY'])
try:
    email = serializer.loads(
        token,
        salt=current_app.config['SECURITY_PASSWORD_SALT'],
        max_age=expiration
    )
except Exception as e :
    return e
return email
```

Once we have our token utility in place, we can now modify user routes. Let's start by disabling user login before email verification. Update the login route to have the following code; here we have changed find_by_username to find_by_email, and now we will expect the user to send the email address in login endpoint JSON data, and if the user isn't verified, we'll return the request with a 400 bad code without the token.

Now your login method should contain the following code.

```
@user_routes.route('/login', methods=['POST'])
def authenticate_user():
      try:
        data = request.get_json()
        if data.get('email') :
            current_user = User.find_by_email(data['email'])
```

```
        elif data.get('username') :
            current_user = User.find_by_username(data['username'])
        if not current_user:
            return response_with(resp.SERVER_ERROR_404)
        if current_user and not current_user.isVerified:
            return response_with(resp.BAD_REQUEST_400)
        if User.verify_hash(data['password'], current_user.
        password):
            access_token = create_access_token(identity =
            current_user.username)
            return response_with(resp.SUCCESS_201,
            value={'message': 'Logged in as {}'.format(current_
            user.username), "access_token": access_token})
        else:
            return response_with(resp.UNAUTHORIZED_401)
      except Exception as e:
        return response_with(resp.INVALID_INPUT_422)
```

Now let's create an endpoint to verify the email token.

We'll start by importing the recently created methods in user.py

```
from api.utils.token import generate_verification_token,
confirm_verification_token
```

Next, add the following GET endpoint to handle email validation right below the user signup method.

```
@user_routes.route('/confirm/<token>', methods=['GET'])
def verify_email(token):
    try:
        email = confirm_verification_token(token)
    except:
        return response_with(resp.SERVER_ERROR_401)
```

```
    user = User.query.filter_by(email=email).first_or_404()
    if user.isVerified:
        return response_with(resp. INVALID_INPUT_422)
    else:
        user.isVerified = True
        db.session.add(user)
        db.session.commit()
        return response_with(resp.SUCCESS_200, value={'message':
        'E-mail verified, you can proceed to login now.'})
```

The next step is to update the user signup method to generate the token and send the email to the specified address for verification, so here we'll start with creating an email utility in our utils to send out emails.

In order to do so, we'll need a flask-mail library; let's start by installing the same. Making sure you are still in the virtual environment, use the following line to install flask-mail in your terminal.

```
(venv) $ pip install Flask-Mail
```

Once installed, let's initiate and configure flask-mail. Add the following variables in config.py to configure mail.

```
MAIL_DEFAULT_SENDER= 'your_email_address'
MAIL_SERVER= 'email_providers_smtp_address'
MAIL_PORT= <mail_server_port>
MAIL_USERNAME= 'your_email_address'
MAIL_PASSWORD= 'your_email_password'
MAIL_USE_TLS= False
MAIL_USE_SSL= True
```

Next create email.py in utils and add the following code.

```
from flask_mail import Message,Mail
from flask import current_app
mail = Mail()
```

Next let's import mail in our main.py and initiate it with app config.

```
from api.utils.email import mail
```

Add this among other imports in main.py, and then right below where we initiated our JWTManager inside create_app, add the following code.

```
    mail.init_app(app)
```

And now our mail object should be initiated with the app config; next in email.py, let's write a method to send out emails.

Add the following code in email.py to create a method send_email which will take the sender's address, subject, and mail template to send.

```
def send_email(to, subject, template):
    msg = Message(
        subject,
        recipients=[to],
        html=template,
        sender=current_app.config['MAIL_DEFAULT_SENDER']
    )
    mail.send(msg)
```

So this is all we need to do in order to send out the verification email; let's go back to users.py and update the user signup method to incorporate the changes.

Let's start by importing the send_email, url_for, and render_template_string method in users.py using the following line.

```
from api.utils.email import send_email
from flask import url_for, render_template_string
```

Update the following code for create_user() method in users.py, right before the return function.

```
try:
    data = request.get_json()
    if(User.find_by_email(data['email']) is not None or
    User.find_by_username(data['username']) is not None):
        return response_with(resp.INVALID_INPUT_422)
    data['password'] = User.generate_hash(data['password'])
    user_schmea = UserSchema()
    user, error = user_schmea.load(data)
    token = generate_verification_token(data['email'])
    verification_email = url_for('user_routes.verify_
    email', token=token, _external=True)
    html = render_template_string("<p>Welcome! Thanks for
    signing up. Please follow this link to activate your
    account:</p> <p><a href='{{ verification_email }}'>{{
    verification_email }}</a></p> <br> <p>Thanks!</p>",
    verification_email=verification_email)
    subject = "Please Verify your email"
    send_email(user.email, subject, html)
    result = user_schmea.dump(user.create()).data
    return response_with(resp.SUCCESS_201)
except Exception as e:
    print e
    return response_with(resp.INVALID_INPUT_422)
```

Here we are supplying the email to generate_verification_token and getting the token in return. Next we use Flask's url_for to generate the verification URL using the verification route we just created and the token. After that we render HTML template using render_template_string of Jinja2 where we supply the HTML string and the verification variable and then we supply all the user-provided email, subject, and HTML to send_email method to send out the verification email.

So this is all we need to setup email verification. Let's start testing out the signup, login, and verification routes to check if everything is working.

Let's start by signup endpoint; open your Postman and request the POST /users API; however, in the JSON body, add a valid email address.

```
{
      "username" : "kunalrelan",
      "password" : "helloworld",
      "email" : "kunal.relan@hotmail.com"
 }
```

We'll use the following JSON in the request data and access the endpoint; the response should be similar to earlier; however, you should get a verification email from your configured mail address on the email you specify in the JSON data with the token (Figure 4-1).

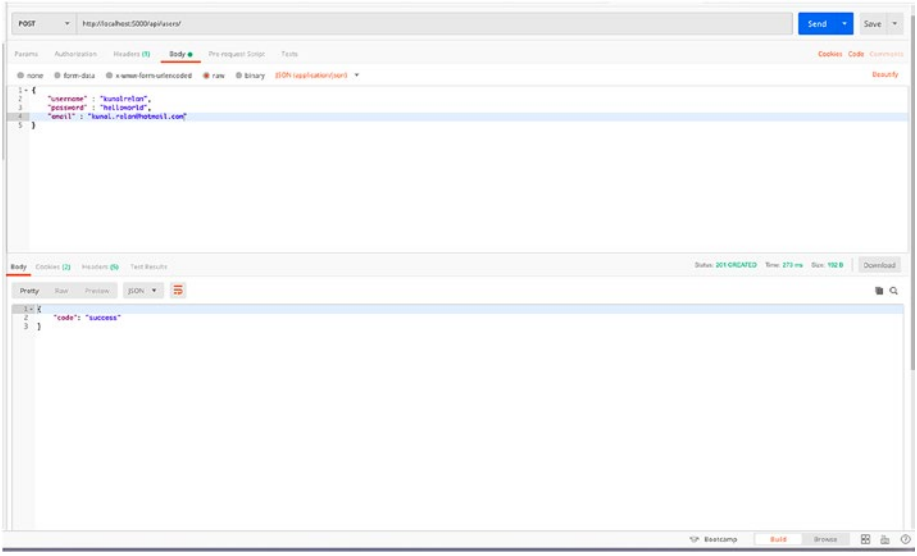

Figure 4-1. *User signup API*

Next, let's check the email inbox to check if the email arrived and verify the user.

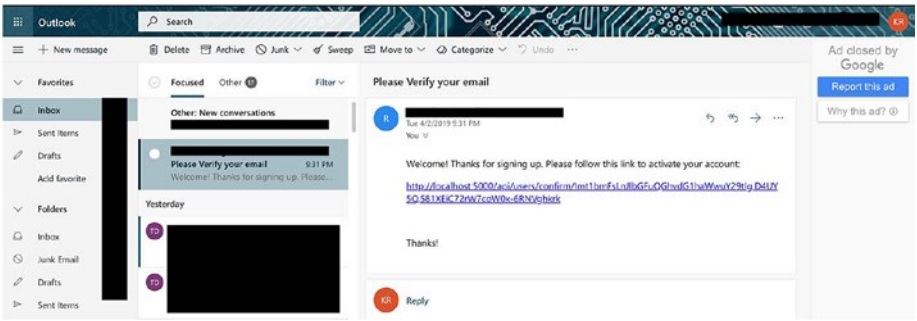

Figure 4-2.

As you can see in Figure 4-2, the verification email arrived with the link to validate the user account. Before we activate the user account, let's try logging in with the user credentials to check if email validation works fine (Figure 4-3).

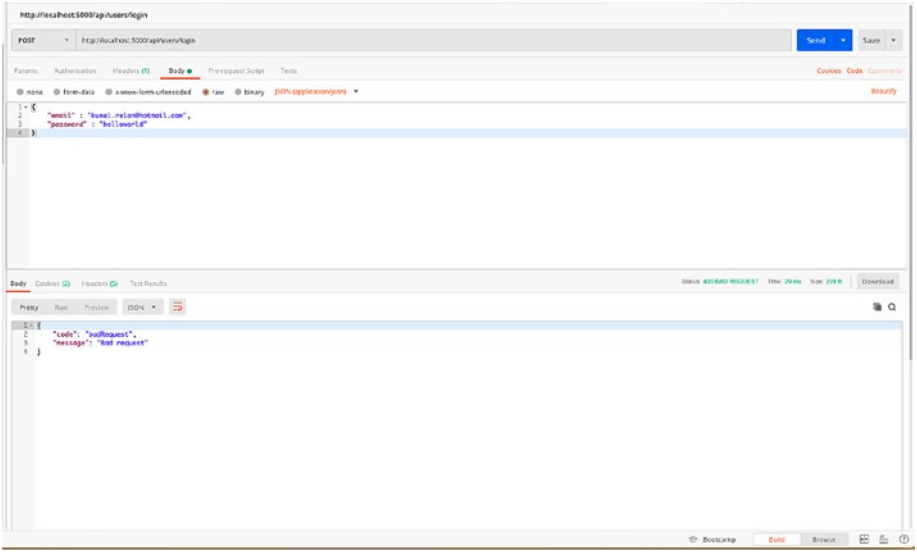

Figure 4-3. *User login without verification*

As you can see in Figure 4-4, the user is not verified and thus can't login. Now let's open the link provided in the email to verify the user which shall then allow the user to login and obtain the JWT token.

Figure 4-4. *User email verification*

Once the user is verified, let's try logging in again, and nowwe should be able to login and obtain the JWT token.

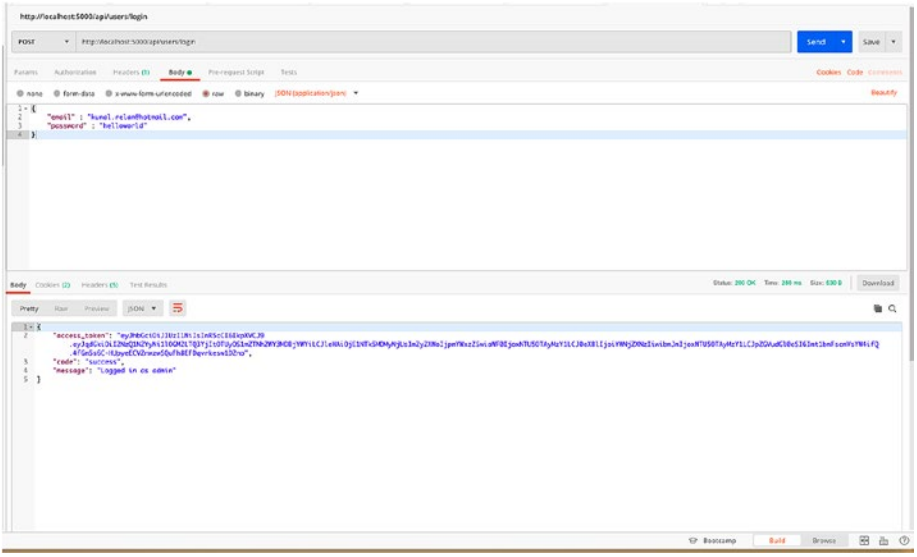

Figure 4-5. *User login after verification*